



uOttawa

Lab 1 - Introduction to VHDL and ASM Design

CEG3155-Digital Systems II

Fall 2026

**School of Electrical Engineering and Computer Science
University of Ottawa**

Course Coordinator:
Prof. Rami Abielmona

Teaching Assistants:
Mathis Hurens Barrette

Group: 11

Student Name and number: Marcel Traore 300379484

Student Name and number: Kai Rasco #300304789

Experiment Date: September 23rd 2026 Submission Date: September 30th 2026

Contents

1	Objective	2
2	Problem Description	2
3	Design	2
3.1	Pseudocode / Flowchart	2
3.2	ASM Chart	3
3.3	Datapath	4
3.4	Detailed ASM Chart	5
3.5	Control Logic (One-FF-Per-State)	6
4	Implementation	7
4.1	Component Descriptions	7
4.2	Top-Level Entity	7
5	Simulation Results	8
5.1	Shift Register	8
5.2	Control Logic	9
5.3	Full Display Controller	9
6	Hardware Test	10
7	Design Obstacles	10
8	Conclusion	10
A	Appendix: VHDL Source Code	11
A.1	lab1_topLevel.vhdl	11
A.2	lab1_control.vhdl	15
A.3	dFF_2.vhdl	17
A.4	dFlipFlop.vhdl	18
A.5	mux4_1bit.vhdl	18
A.6	mux4_nBit.vhdl	19
A.7	shiftReg-nbit.vhdl	19

1 Objective

The objective of this lab is to design a light display controller using the ASM method, implement it in structural VHDL and verify it by simulation.

2 Problem Description

The goal is a controller for a row of eight LEDs driven by two switches. When only LEFT is on, a single light moves from right to left. When only RIGHT is on, a single light moves from left to right. When both are on, both lights move at the same time and cross each other. When both are off, the display is blank, but the position of each light is kept, so turning a switch back on resumes the motion from where it stopped. GReset brings the whole circuit back to its initial state. On a board, the lights would move one position per second so the motion is visible. As agreed with the TA, this lab is evaluated on the simulation of the top-level entity only, so the design is clocked directly by GClock (Section 4.2).

Port	Name	Description
Input	GClock	Global clock
Input	GReset	Global reset
Input	Right	Right switch: 0 = off, 1 = on
Input	Left	Left switch: 0 = off, 1 = on
Output	DisplayOut[7..0]	Eight display LEDs

Table 1: Input/output specification

3 Design

3.1 Pseudocode / Flowchart

The design starts from the pseudocode given in the lab manual:

- $DISPLAY \leftarrow 00000000$; $LMASK \leftarrow 00000001$; $RMASK \leftarrow 10000000$;
- if (LEFT and RIGHT)
 - $DISPLAY \leftarrow LMASK \vee RMASK$;
 - $LMASK \leftarrow LMASK \ll 1$;
 - $RMASK \leftarrow RMASK \gg 1$;
- else if (LEFT)
 - $DISPLAY \leftarrow LMASK$;
 - $LMASK \leftarrow LMASK \ll 1$;
- else if (RIGHT)
 - $DISPLAY \leftarrow RMASK$;
 - $RMASK \leftarrow RMASK \gg 1$;
- else
 - $DISPLAY \leftarrow 00000000$;
- Go back to first if statement;

The ASM chart in Figure 1 also serves as the flowchart of the solution.

3.2 ASM Chart

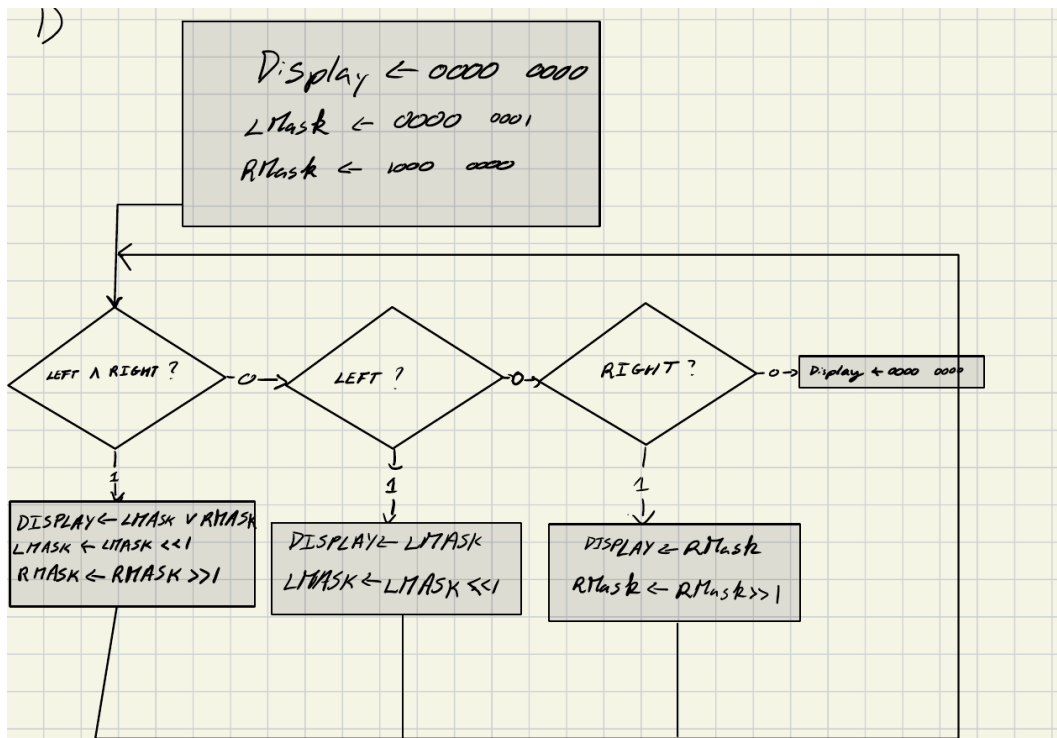


Figure 1: ASM chart

The chart has five states. S0 is the initial state: DISPLAY is cleared, LMASK is set to 00000001 and RMASK to 10000000. From every state the chart returns to the same chain of decisions. If both switches are on (S1), DISPLAY takes LMASK OR RMASK and both masks shift. If only LEFT is on (S2), DISPLAY takes LMASK and LMASK shifts left. If only RIGHT is on (S3), DISPLAY takes RMASK and RMASK shifts right. If neither is on (S4), DISPLAY is cleared. The masks are not reset in S4, so the lights resume from where they stopped.

3.3 Datapath

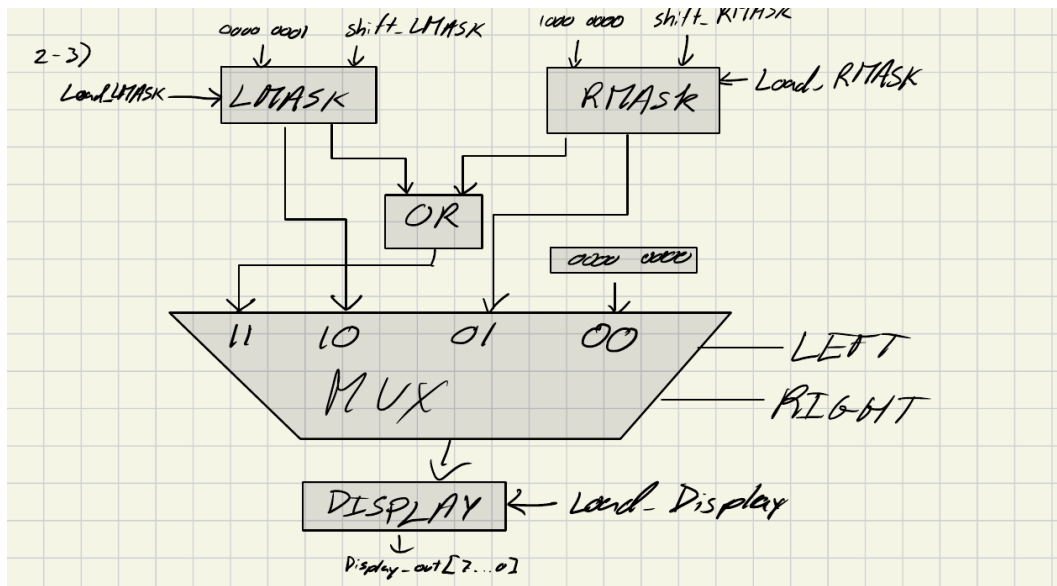


Figure 2: Datapath

Following the ASM rules, each unique name in the RTN statements gets its own register: LMASK, RMASK and DISPLAY. LMASK and RMASK are shift registers with a parallel load of their initial value (00000001 and 10000000) and control inputs Load_LMASK/shift_LMASK and Load_RMASK/shift_RMASK. An 8-bit OR gate forms LMASK OR RMASK. Since DISPLAY has four possible sources, a 4-to-1 mux selects its input, with LEFT and RIGHT used directly as the select lines: 11 selects LMASK OR RMASK, 10 selects LMASK, 01 selects RMASK and 00 selects 00000000. DISPLAY is loaded when Load_Display is high and drives DisplayOut[7..0].

Because the mux select comes directly from the switches, loading DISPLAY in S0 does not guarantee 00000000 unless both switches are off. In the top-level entity this is handled by the global reset: GReset clears DISPLAY, LMASK and RMASK through the asynchronous reset of their flip-flops (see Section 4.2).

3.4 Detailed ASM Chart

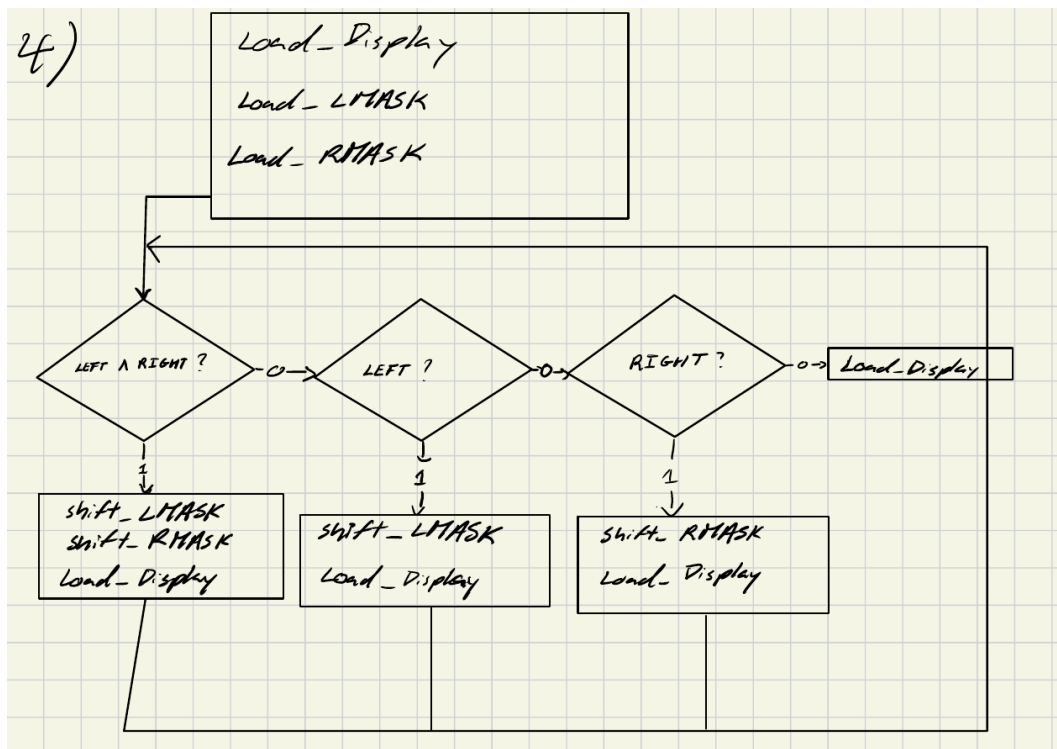


Figure 3: Detailed ASM chart

Each RTN statement of Figure 1 is replaced by the control signals that perform it on the datapath of Figure 2, and each decision box tests the LEFT and RIGHT status signals. S0 asserts Load_Display, Load_LMASK and Load_RMASK. S1 asserts shift_LMASK, shift_RMASK and Load_Display. S2 asserts shift_LMASK and Load_Display, S3 asserts shift_RMASK and Load_Display, and S4 only asserts Load_Display (the mux already selects 00000000). Unlisted signals are 0.

3.5 Control Logic (One-FF-Per-State)

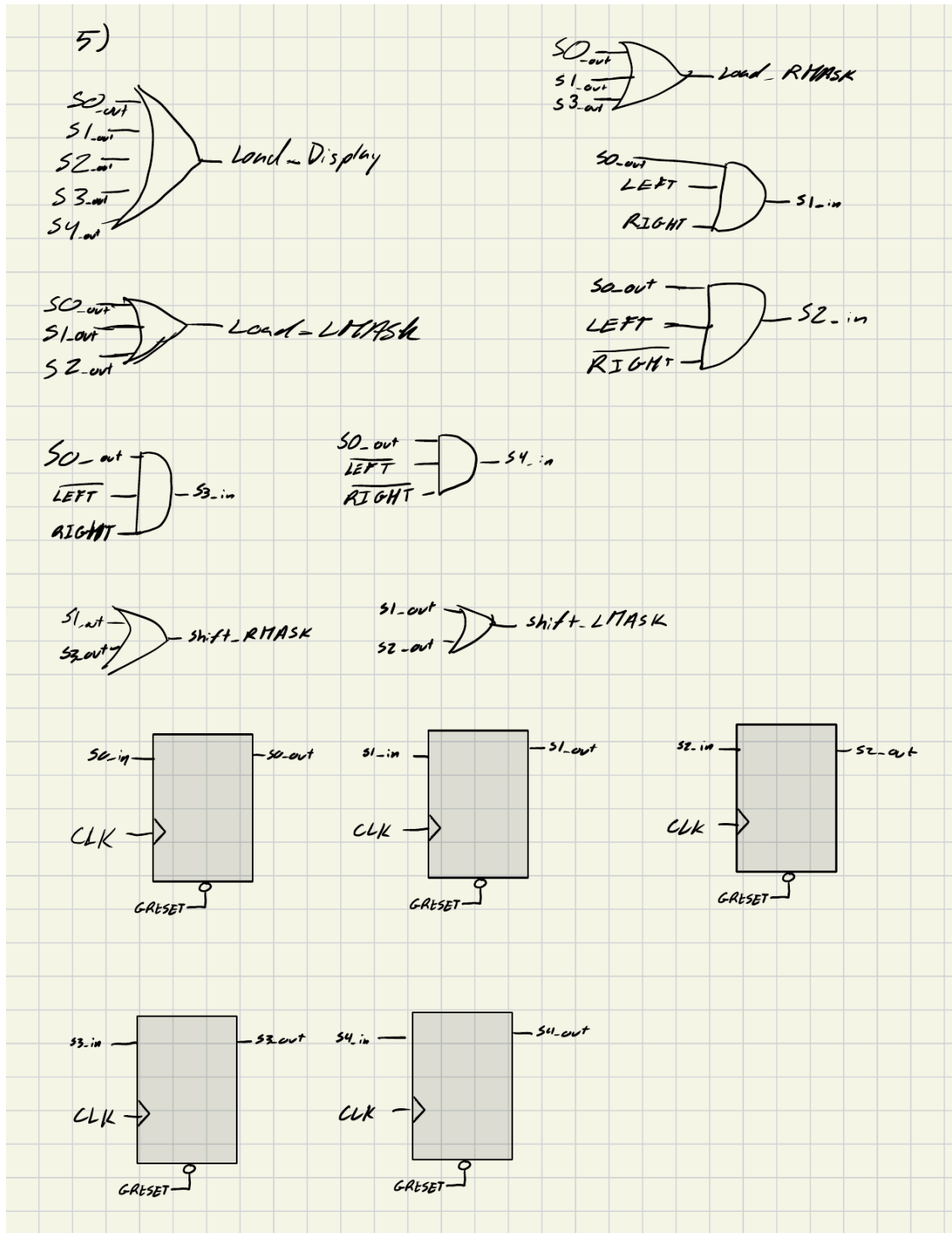


Figure 4: Control logic (one-FF-per-state)

One D flip-flop is used per state (S0 to S4), all clocked by CLK, so exactly one state is active at a time. Each control signal is the OR of the states that assert it in Figure 3:

- $Load_Display = S0 + S1 + S2 + S3 + S4$
- $Load_LMASK = S0$, $shift_LMASK = S1 + S2$

- $\text{Load_RMask} = S0, \quad \text{shift_RMask} = S1 + S3$

Every state returns to the same decision boxes, and exactly one state is always active, so the next-state inputs only depend on the switches:

- $S1_in = \text{LEFT} \cdot \text{RIGHT}$
- $S2_in = \text{LEFT} \cdot \text{RIGHT}'$
- $S3_in = \text{LEFT}' \cdot \text{RIGHT}$
- $S4_in = \text{LEFT}' \cdot \text{RIGHT}'$

Figure 4 was drawn before the VHDL was written, and it differs from the equations above in two places: it ANDs $S0_out$ into each next-state input, and it ORs $S0$, $S1$ and $S2$ into Load_LMask (and $S0$, $S1$ and $S3$ into Load_RMask). The equations above are the ones implemented in the VHDL.

In the VHDL (`lab1_control`), the state flip-flops are `dFF_2`, which has no reset input, so `GReset` is handled synchronously: $S0_in = GReset$, and each of $S1_in$ to $S4_in$ is also ANDed with $GReset'$, so the controller is in $S0$ while `GReset` is high and leaves it on the first clock edge after `GReset` is released. The mask registers use the `shiftRegister` component, where load has priority over shift. The VHDL drives the load inputs with $S0$ only and the shift inputs with $S1 + S2$ and $S1 + S3$, exactly as in the equations above.

4 Implementation

4.1 Component Descriptions

The following components were written in VHDL:

- **dFF_2**: D flip-flop (course-provided, no reset) used as the one-hot state register in the control path.
- **dFlipFlop**: positive-edge-triggered D flip-flop built from six NAND-type gates, with an asynchronous active-low reset ($i_reset_bar = 0$ forces Q to 0). It is the storage element of the shift register.
- **mux4_1bit**: 1-bit 4-to-1 multiplexer written as a sum of products; select 00 picks a , 01 picks b , 10 picks c and 11 picks d .
- **shiftRegister**: generic n -bit register with one `mux4_1bit` and one `dFlipFlop` per bit. The select lines are $sel(1) = \text{shiftL} + \text{load}$ and $sel(0) = \text{shiftR} + \text{load}$, so 00 holds, 01 shifts right (bit i takes bit $i+1$), 10 shifts left (bit i takes bit $i-1$) and 11 loads d . Because load drives both select lines, it has priority over either shift. The end bits take `shiftExtension` as the incoming bit. It is intended for `LMask` and `RMask`.
- **lab1_control**: one-hot controller with states $S0$ to $S4$ producing the load and shift control signals.
- **mux4_nBit**: generic n -bit 4-to-1 multiplexer (one sum-of-products expression per bit), used as the 8-bit `DISPLAY` input mux.
- **lab1_topLevel**: top-level entity connecting the controller to the datapath (Section 4.2).

The full code is in the Appendix.

4.2 Top-Level Entity

`lab1_topLevel` has the ports of the I/O specification (`GClock`, `GReset`, `Left`, `Right`, `DisplayOut[7..0]`) and is built structurally from the components above:

- **control (lab1_control)**: receives `Left`, `Right` and the clock, and drives `Load_Display`, `Load_LMask`, `Load_RMask`, `shift_LMask` and `shift_RMask`.
- **IMaskReg and rMaskReg (shiftRegister, n = 8)**: `LMask` and `RMask`, with parallel inputs fixed at 00000001 and 10000000. `LMask` only uses `shiftL` and `RMask` only uses `shiftR`. Each register's `shiftExtension` is tied to its own opposite end bit (`LMask(7)` for `LMask`, `RMask(0)` for `RMask`), so

the masks rotate instead of shifting out: after reaching the last LED the light wraps back to the first one and keeps moving.

- **OR gate:** the 8-bit expression LMASK or RMASK forms the “both” input of the mux.
- **mux (mux4_nBit):** sel(1) = Left and sel(0) = Right, with inputs 00000000, RMASK, LMASK and LMASK OR RMASK, as in Figure 2.
- **displayReg (shiftRegister, n = 8):** the DISPLAY register. Its shift inputs are tied to 0, so it only loads (when Load_Display is high) or holds, and its output drives DisplayOut.

GReset is active high. It is inverted once and sent to the controller (which inverts it again, so S0_in = GReset) and to the active-low reset of all three registers. While GReset is high, DISPLAY, LMASK and RMASK are held at 00000000 and the controller is in S0. On the first clock edge after GReset is released, S0 is still active, so the masks load 00000001 and 10000000 and the controller moves to S1 to S4 according to the switches.

Since the design is only verified in simulation, no clock divider is used: GClock drives the controller and the registers directly, so the lights move one position per clock cycle. The course clock divider (clk_div) is declared in the top-level but its instance is commented out; it would only be needed to slow the lights down to about 1 Hz on a board.

5 Simulation Results

5.1 Shift Register

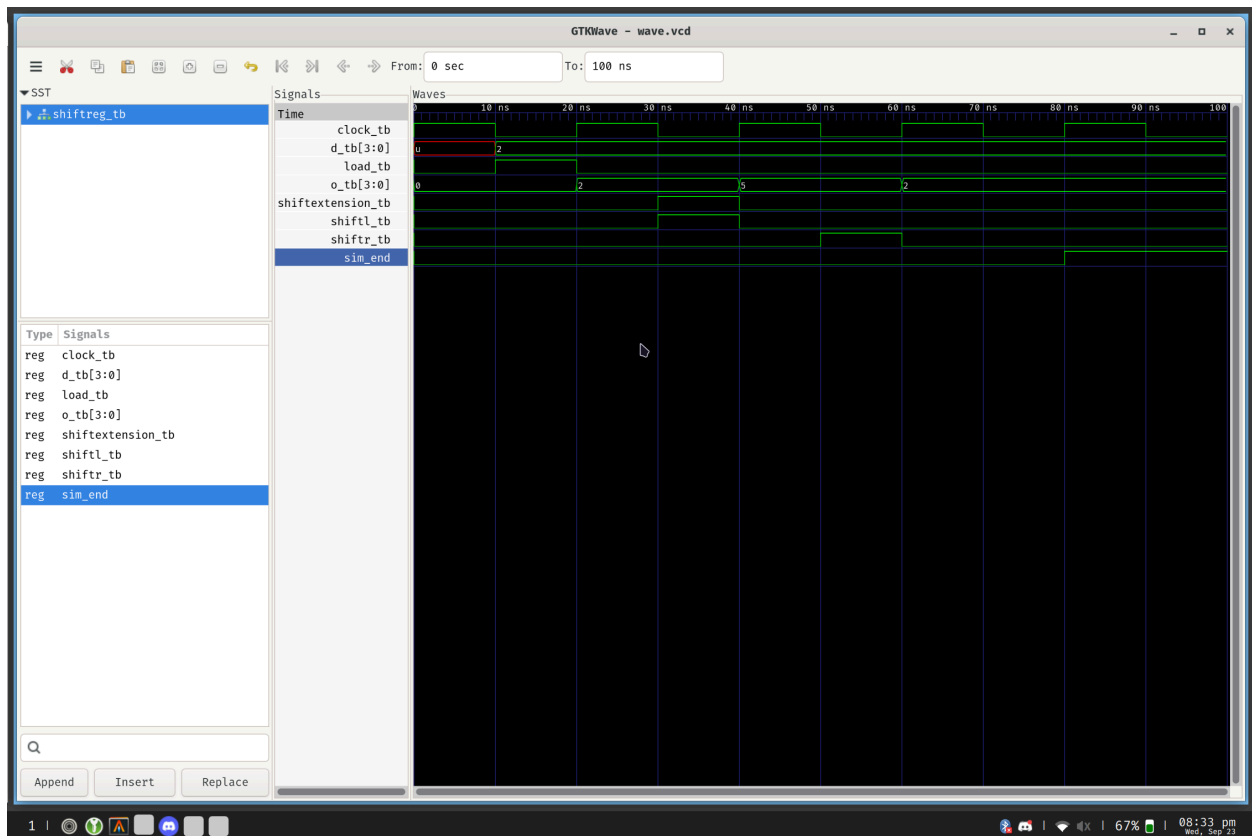


Figure 5: Shift register simulation (4 bits, clock period 20 ns)

The testbench checks the load and shift left operations of the shift register. At 10 ns, `d_tb` is set to 0010 and `load_tb` goes high, so at the rising edge at 20 ns the register loads 0010. From 30 to 40 ns, `shiftl_tb` and `shiftextension_tb` are high, so at the 40 ns edge the register shifts left and the extension bit enters bit 0: 0010 becomes 0101. Between these operations no control signal is high, so the output holds its value. The results match the expected values.

5.2 Control Logic

The controller is not simulated on its own; it is verified through the top-level simulation in Figure 6. Tracing the VHDL equations: while `GReset` is high, only `S0` is set on the next clock edge, which asserts `Load_LMASK`, `Load_RMASK` and `Load_Display`. After `GReset` is released, each switch combination sets exactly one of `S1` (LEFT and RIGHT), `S2` (LEFT only), `S3` (RIGHT only) or `S4` (neither) on the next edge, which matches the detailed ASM chart.

5.3 Full Display Controller

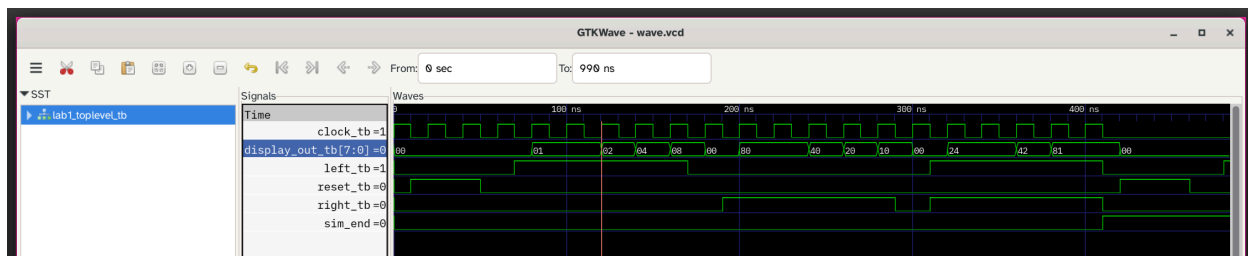


Figure 6: Top-level simulation

The testbench `lab1_topLevel_TB` drives `GClock` directly (no clock divider) with a 20 ns period, so the rising edges are at multiples of 20 ns. `GReset` is high from 10 to 50 ns. LEFT alone is then on from 70 to 170 ns, RIGHT alone from 190 to 290 ns, and both from 310 to 410 ns, with both switches off in between. Table 2 lists `DisplayOut` (in hex) at each rising edge.

Phase	Edges (ns)	DisplayOut
Reset	20 to 60	00, 00, 00
LEFT only	80 to 160	01, 01, 02, 04, 08
Both off	180	00
RIGHT only	200 to 280	80, 80, 40, 20, 10
Both off	300	00
Both on	320 to 400	24, 24, 42, 81, 81

Table 2: `DisplayOut` in the top-level simulation

`DisplayOut` stays at 00 during reset, and `LMASK` and `RMASK` load their initial values on the 60 ns edge. With LEFT on, the light moves from right to left (01, 02, 04, 08). With RIGHT on, it moves from left to right (80, 40, 20, 10). With both switches off, the display is blank. With both on, `DisplayOut` is `LMASK OR RMASK` and both lights move at the same time. At 410 ns, `sim_end` stops the clock, which ends the test. After that, the stimulus process starts over because it does not end with a `wait;` statement, so `GReset` is raised again and clears the display.

The waveform also shows the effect of a one-clock delay. The mux select comes directly from the switches, but the shift signals come from the state register, which only follows the switches one edge later:

- **Start of a phase:** on the first edge after a switch turns on, the state is still S4, so DISPLAY loads the mask without shifting it. The first value of each phase (01, 80, 24) is therefore shown for two cycles.
- **End of a phase:** on the first edge after a switch turns off, the state is still S2 or S3, so the mask shifts once more while DISPLAY loads 00. LMASK ends at 00100000 instead of 00010000, and RMASK at 00000100 instead of 00001000. When both switches turn on, each light therefore resumes one position past where it was last shown: $24 = 00100000 \text{ OR } 00000100$.

Because the lights had already passed each other by then, they move apart in the last phase (24, 42, 81) instead of toward each other. At 81 the lights are at the two ends. On the next shift, the masks rotate (LMASK 10000000 becomes 00000001 and RMASK 00000001 becomes 10000000), so 81 is shown again. Apart from these one-clock effects, which would not be noticeable at 1 Hz, the simulation matches the expected behaviour.

6 Hardware Test

As agreed with the TA, the design is evaluated on the top-level simulation (Figure 6) only, so it was not synthesized or demonstrated on the board.

7 Design Obstacles

- **No reset on the state flip-flops:** the course-provided dFF_2 has no reset input, so GReset was made synchronous by feeding it into S0_in and blocking S1_in to S4_in while it is high.
- **Load and shift in the same state:** the chart loads the masks in S0 while S1 to S3 shift them. Giving load priority over shift in shiftRegister let the shift signals stay simple without conflicting with the load in S0.
- **Reset of the datapath:** loading DISPLAY in S0 does not clear it unless both switches are off, since the mux select comes from the switches. This was solved by connecting GReset to the asynchronous reset of all three registers.

8 Conclusion

The light display controller was designed with the five steps of the ASM method: pseudocode, ASM chart, datapath, detailed ASM chart and one-FF-per-state control logic. The control path, the shift register, the datapath and the top-level entity were implemented in structural VHDL. The shift register simulation matched the expected load and shift left results, and the top-level simulation shows the lights moving left, right, and both at once according to the switches, with the masks kept while both switches are off. The only difference from the ideal behaviour is a one-clock delay between the switches and the state register, which makes the first value of each phase last two cycles and makes the lights resume one position later.

A Appendix: VHDL Source Code

A.1 lab1_topLevel.vhdl

```
library ieee;
use ieee.std_logic_1164.all;
entity lab1_topLevel is
  port (
    GClock : in std_logic;
    GReset : in std_logic;
    Left : in std_logic;
    Right : in std_logic;
    DisplayOut: out std_logic_vector(7 downto 0)
  );
end lab1_topLevel;

architecture structural of lab1_topLevel is

  component shiftRegister is
    generic ( n : integer := 4 );
    port (
      clock, reset : in std_logic;
      load, shiftL, shiftR : in std_logic;
      shiftExtension : in std_logic;

      d : in std_logic_vector(n-1 downto 0);
      o : out std_logic_vector(n-1 downto 0)
    );
  end component;

  component mux4_nBit is
    generic (bit_width : integer := 8);
    port (
      i_1, i_2, i_3, i_4 : in std_logic_vector(bit_width-1 downto 0);
      sel : in std_logic_vector(1 downto 0);
      o : out std_logic_vector(bit_width-1 downto 0)
    );
  end component;

  component lab1_control is
    port (
      i_clock: in std_logic;
      i_reset: in std_logic;
      i_left : in std_logic;
      i_right : in std_logic;

      o_loadDisplay : out std_logic;
      o_loadLMask: out std_logic;
      o_loadRMask : out std_logic;
      o_shiftLMask: out std_logic;
      o_shiftRMask : out std_logic
    );
  end component;
```

```

component clk_div IS
  port (
    clock_25mhz      : in  std_logic;
    clock_1mhz       : out std_logic;
    clock_100khz     : out std_logic;
    clock_10khz      : out std_logic;
    clock_1khz       : out std_logic;
    clock_100hz      : out std_logic;
    clock_10hz       : out std_logic;
    clock_1hz        : out std_logic);

end component;

signal zero : std_logic;
signal zero_8bit : std_logic_vector(7 downto 0);

signal int_loadDisplay, int_loadLMask, int_loadRMask : std_logic;
signal int_shiftRmask, int_shiftLmask : std_logic;

signal int_displayIn, int_rmaskIn, int_lmaskIn : std_logic_vector(7 downto 0);
signal int_displayOut, int_rmaskOut, int_lmaskOut, int_bothOut : std_logic_vector(7 downto 0);

signal int_muxSel : std_logic_vector(1 downto 0);
signal int_clock, int_reset: std_logic;

begin

zero <= '0';
zero_8bit <= "00000000";

int_bothOut <= int_lmaskOut or int_rmaskOut;
int_muxSel(0) <= Right;
int_muxSel(1) <= Left;

int_lmaskIn <= "00000001";
int_rmaskIn <= "10000000";

-- clock_div: clk_div
-- port map (
--   clock_25mhz => GClock,
--   clock_1hz => int_clock
-- );
int_clock <= GClock;
int_reset <= not GReset;

control : lab1_control
  port map(
    i_clock => int_clock,
    i_reset => int_reset,
    i_left => Left,
    i_right => Right,
    o_loadDisplay => int_loadDisplay,
    o_loadLMask => int_loadLMask,
    o_loadRMask => int_loadRMask,
    o_shiftLMask => int_shiftLmask,

```

```

        o_shiftRMask => int_shiftRmask
    );

mux : mux4_nBit
    generic map( bit_width => 8)
    port map(
        i_1 => zero_8bit,
        i_2 => int_rmaskOut,
        i_3 => int_lmaskOut,
        i_4 => int_bothOut,
        sel => int_muxSel,
        o => int_displayIn
    );

displayReg : shiftRegister
    generic map( n => 8 )
    port map(
        clock => int_clock,
        reset => int_reset,
        load => int_loadDisplay,
        shiftL => zero,
        shiftR => zero,
        shiftExtension => zero,
        d => int_displayIn,
        o => DisplayOut
    );

lMaskReg : shiftRegister
    generic map( n => 8 )
    port map(
        clock => int_clock,
        reset => int_reset,
        load => int_loadLMask,
        shiftL => int_shiftLmask,
        shiftR => zero,
        shiftExtension => int_lmaskOut(7),
        d => int_lmaskIn,
        o => int_lmaskOut
    );

rMaskReg : shiftRegister
    generic map( n => 8 )
    port map(
        clock => int_clock,
        reset => int_reset,
        load => int_loadRMask,
        shiftL => zero,
        shiftR => int_shiftRmask,
        shiftExtension => int_rmaskOut(0),
        d => int_rmaskIn,
        o => int_rmaskOut
    );

end architecture;

entity lab1_topLevel_TB is

```

```

end lab1_topLevel_TB;

library ieee;
use ieee.std_logic_1164.all;
architecture structural of lab1_topLevel_TB is
component lab1_topLevel is
    port (
        GClock : in std_logic;
        GReset : in std_logic;
        Left : in std_logic;
        Right : in std_logic;
        DisplayOut: out std_logic_vector(7 downto 0)
    );
end component;

    signal clock_tb, reset_tb, left_tb, right_tb : std_logic;
    signal display_out_tb: std_logic_vector(7 downto 0);
    signal sim_end : BOOLEAN := false;
    constant CLOCK_PERIOD : time := 20 ns;

begin
dut: lab1_topLevel
    port map(
        GClock => clock_tb,
        GReset => reset_tb,
        Left => left_tb,
        Right => right_tb,
        DisplayOut => display_out_tb
    );

    clock_process:
    process begin
        while (not sim_end) loop
            clock_tb <= '1';
            wait for clock_period / 2;
            clock_tb <= '0';
            wait for clock_period / 2;
        end loop;
        wait;
    end process clock_process;

    stimulus:
    process begin

        reset_tb <= '0';
        left_tb <= '0';
        right_tb <= '0';

        wait for clock_period / 2;
        reset_tb <= '1';

        wait for clock_period;
        wait for clock_period;
        reset_tb <= '0';

```

```

wait for clock_period;
left_tb <= '1';
wait for clock_period;
wait for clock_period;
wait for clock_period;
wait for clock_period;
wait for clock_period;
left_tb <= '0';

wait for clock_period;
right_tb <= '1';
wait for clock_period;
wait for clock_period;
wait for clock_period;
wait for clock_period;
wait for clock_period;
right_tb <= '0';

wait for clock_period;
left_tb <= '1';
right_tb <= '1';
wait for clock_period;
wait for clock_period;
wait for clock_period;
wait for clock_period;
wait for clock_period;
left_tb <= '0';
right_tb <= '0';

sim_end <= true;

end process stimulus;

end architecture;

```

A.2 lab1_control.vhdl

```

library ieee;
use ieee.std_logic_1164.all;

entity lab1_control is
  port (
    i_clock: in std_logic;
    i_reset: in std_logic;
    i_left : in std_logic;
    i_right : in std_logic;

    o_loadDisplay : out std_logic;
    o_loadLMask: out std_logic;
    o_loadRMask : out std_logic;
    o_shiftLMask: out std_logic;
    o_shiftRMask : out std_logic
  );

```

```

end lab1_control;

architecture structural of lab1_control is
    signal s0in, slin, s2in, s3in, s4in : std_logic;
    signal s0out, slout, s2out, s3out, s4out : std_logic;

    component dFF_2 is
        port(
            i_d   : in  std_logic;
            i_clock : in  std_logic;
            o_q, o_qbar : out std_logic);
    end component;

    signal int_reset : std_logic;

begin
    int_reset <= not i_reset;

    s0in <= int_reset;
    slin <= i_left and i_right and not int_reset;
    s2in <= i_left and not i_right and not int_reset;
    s3in <= not i_left and i_right and not int_reset;
    s4in <= not i_left and not i_right and not int_reset;

    o_loadLMask <= s0out;
    o_loadRMask <= s0out;

    o_loadDisplay <= s0out or slout or s2out or s3out or s4out;
    o_shiftLMask <= slout or s2out;
    o_shiftRMask <= slout or s3out;

    s0 : dFF_2
    port map (
        i_d => s0in,
        i_clock => i_clock,
        o_q => s0out
    );

    s1 : dFF_2
    port map (
        i_d => slin,
        i_clock => i_clock,
        o_q => slout
    );

    s2 : dFF_2
    port map (
        i_d => s2in,
        i_clock => i_clock,
        o_q => s2out
    );

    s3 : dFF_2
    port map (
        i_d => s3in,
        i_clock => i_clock,

```

```

        o_q => s3out
    );

s4 : dFF_2
port map (
    i_d => s4in,
    i_clock => i_clock,
    o_q => s4out
);

end architecture;

```

A.3 dFF_2.vhdl

```

-----
-- Title       : Type D Flip-Flop - 2nd realization
-- Project     : VHDL Synthesis Overview
-----
-- File        : dFF_2.vhd
-- Author      : Rami Abielmona <rabiemo@site.uottawa.ca>
-- Created     : 2003/05/17
-- Last modified : 2007/09/25
-----
-- Description : This file creates a flip-flop of type D as defined in the VHDL
--               Synthesis lecture. The architecture is done at the RTL
--               abstraction level and the implementation is done in structural
--               VHDL.
-----
-- Modification history :
-- 2003.05.17  R. Abielmona    Creation
-- 2004.09.22  R. Abielmona    Ported for CEG 3550
-- 2007.09.25  R. Abielmona    Modified copyright notice
-----
-- This file is copyright material of Rami Abielmona, Ph.D., P.Eng., Chief Research
-- Scientist at Larus Technologies. Permission to make digital or hard copies of part
-- or all of this work for personal or classroom use is granted without fee
-- provided that copies are not made or distributed for profit or commercial
-- advantage and that copies bear this notice and the full citation of this work.
-- Prior permission is required to copy, republish, redistribute or post this work.
-- This notice is adapted from the ACM copyright notice.
-----

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY dFF_2 IS
    PORT(
        i_d      : IN  STD_LOGIC;
        i_clock   : IN  STD_LOGIC;
        o_q, o_qBar : OUT STD_LOGIC);
END dFF_2;

ARCHITECTURE rtl OF dFF_2 IS
    SIGNAL int_q : STD_LOGIC;

```

```

BEGIN

oneBitRegister:
PROCESS(i_clock)
BEGIN
  IF (i_clock'EVENT and i_clock = '1') THEN
    int_q <= i_d;
  END IF;
END PROCESS oneBitRegister;

  -- Output Driver

  o_q <= int_q;
  o_qBar <= not(int_q);

END rtl;

```

A.4 dFlipFlop.vhdl

```

library ieee;
use ieee.std_logic_1164.all;
entity dFlipFlop is
  port (
    i_d      : in  std_logic;
    i_clk     : in  std_logic;
    i_reset_bar : in std_logic;
    o_q, o_qBar : out std_logic
  );
end dFlipFlop;

architecture structural of dFlipFlop is
  signal a : std_logic := '0';
  signal b, c : std_logic := '1';
  signal d : std_logic := '1';
  signal int_q : std_logic := '0';
  signal int_qBar : std_logic := '1';
begin
  a <= d nand b;
  b <= not (a and i_clk and i_reset_bar);
  c <= not (b and d and i_clk);
  d <= not (c and i_d and i_reset_bar) after 0.5 ns;

  int_q <= b nand int_qBar;
  int_qBar <= not (int_q and c and i_reset_bar);

  o_q <= int_q;
  o_qBar <= int_qBar;
end architecture;

```

A.5 mux4_1bit.vhdl

```

-- ===== 4x1 MUX
library ieee;
use ieee.std_logic_1164.all;

```

```

entity mux4_1bit is
    port (
        a, b, c, d : in std_logic;
        sel : in std_logic_vector(1 downto 0);
        o : out std_logic
    );
end mux4_1bit;

architecture structural of mux4_1bit is
    -- s1, s0
    -- a for 00
    -- b for 01
    -- c for 10
    -- d for 11
begin
    o <= (a and (not sel(1)) and (not sel(0))) or
        (b and (not sel(1) and sel(0))) or
        (c and sel(1) and (not sel(0))) or
        (d and sel(1) and sel(0));
end structural;

```

A.6 mux4_nBit.vhdl

```

-- ==== 4 INPUT N-BIT MUX
-- deps: none
library ieee;
use ieee.std_logic_1164.all;
entity mux4_nBit is
    generic (bit_width : integer := 8);
    port (
        i_1, i_2, i_3, i_4 : in std_logic_vector(bit_width-1 downto 0);
        sel : in std_logic_vector(1 downto 0);
        o : out std_logic_vector(bit_width-1 downto 0)
    );
end mux4_nBit;

architecture rtl of mux4_nBit is
begin
    b_i: for i in bit_width-1 downto 0 generate
    begin
        o(i) <=
            (i_1(i) and not sel(1) and not sel(0)) or
            (i_2(i) and not sel(1) and sel(0)) or
            (i_3(i) and sel(1) and not sel(0)) or
            (i_4(i) and sel(1) and sel(0));
    end generate;
end rtl;

```

A.7 shiftReg-nbit.vhdl

```

-- ==== N-BIT SHIFT REGISTER
library ieee;
use ieee.std_logic_1164.all;

```

```

entity shiftRegister is
  generic ( n : integer := 4 );
  port (
    clock, reset : in std_logic;
    load, shiftL, shiftR : in std_logic;
    shiftExtension : in std_logic;

    d : in std_logic_vector(n-1 downto 0);
    o : out std_logic_vector(n-1 downto 0)
  );
end shiftRegister;

architecture structural of shiftRegister is
  signal int_d : std_logic_vector(n-1 downto 0);
  signal int_q : std_logic_vector(n-1 downto 0);
  signal int_sel : std_logic_vector(1 downto 0);

  component dFlipFlop is
    port (
      i_d      : in std_logic;
      i_clk     : in std_logic;
      i_reset_bar : in std_logic;
      o_q, o_qBar : out std_logic
    );
  end component;

  component mux4_1bit is
    port (
      a, b, c, d : in std_logic;
      sel : in std_logic_vector(1 downto 0);
      o : out std_logic
    );
  end component;
begin

  o <= int_q;

  -- == MUXES have to separate out LSB and MSB to add optional shift extension
  -- LOAD takes priority over shift
  int_sel(1) <= shiftL or load;
  int_sel(0) <= shiftR or load;

  muxes: for i in n-2 downto 1 generate
  begin
    mux_i : mux4_1bit
    port map (
      a => int_q(i), -- hold
      b => int_q(i+1), -- shift L
      c => int_q(i-1), -- shift R
      d => d(i), -- load
      sel => int_sel,
      o => int_d(i)
    );
  end generate;

```

```

-- rightmost
mux_lsb: mux4_lbit
port map (
  a => int_q(0),
  b => int_q(1),
  c => shiftExtension,
  d => d(0),
  sel => int_sel,
  o => int_d(0)
);

-- leftmost
mux_msb: mux4_lbit
port map (
  a => int_q(n-1),
  b => shiftExtension,
  c => int_q(n-2),
  d => d(n-1),
  sel => int_sel,
  o => int_d(n-1)
);

bits: for i in n-1 downto 0 generate
begin
  b_i : dFlipFlop
  port map (
    i_clk => clock,
    i_reset_bar => reset,
    i_d => int_d(i),
    o_q => int_q(i)
  );
end generate;

end structural;

-- ===== SHIFT REGISTER TEST BENCH
library ieee;
use ieee.std_logic_1164.all;
entity shiftReg_TB is
  generic ( n : integer := 4 );
end shiftReg_TB;

architecture testbench of shiftReg_TB is
  signal clock_tb : std_logic := '0';
  signal d_tb, o_tb: std_logic_vector(n-1 downto 0);
  signal load_TB, shiftL_TB, shiftR_TB, shiftExtension_TB: std_logic := '0';
  signal sim_end : BOOLEAN := false;
  constant CLOCK_PERIOD : time := 20 ns;

  component shiftRegister is
    generic ( n : integer := 4 );
    port (
      clock, reset : in std_logic;
      load, shiftL, shiftR : in std_logic;
      shiftExtension : in std_logic;

```

```

        d : in std_logic_vector(n-1 downto 0);
        o : out std_logic_vector(n-1 downto 0)
    );
    end component;
begin
    dut: shiftRegister
    port map (
        clock => clock_tb,
        load => load_TB,
        shiftL => shiftL_tb,
        shiftR => shiftR_tb,
        shiftExtension => shiftExtension_tb,

        d => d_tb,
        o => o_tb,

        reset => '1'
    );

    clock_process:
    process begin
        while (not sim_end) loop
            clock_tb <= '1';
            wait for clock_period / 2;
            clock_tb <= '0';
            wait for clock_period / 2;
        end loop;
        wait;
    end process clock_process;

    stimulus:
    process begin
        wait for clock_period / 2;
        d_tb <= "0010"; load_TB <= '1';
        wait for clock_period / 2; -- clock rising edge
        load_TB <= '0';
        wait for clock_period / 2; -- falling
        assert o_tb = "0010" report "load failed";

        shiftL_TB <= '1'; shiftExtension_TB <= '1';
        wait for clock_period / 2; -- rising
        shiftL_TB <= '0'; shiftExtension_TB <= '0';
        wait for clock_period / 2; -- rising
        assert o_tb = "0101" report "shift failed";

        wait for clock_period / 2;
        sim_end <= true;
        wait;
    end process stimulus;
end testbench;

```